

Sam Mengel: Section 01

Question 1: Heuristic design and admissibility [15 points]

For each of the three cost measures— steps, Scrabble, and frequency—describe the heuristic implemented in your program.

For each heuristic:

1. Give a precise formula or pseudocode.

The clue that points to the algorithm's completion is the goal state, which is self.goal . Because the algorithm knows what the goal state is and how far off it is from the goal state given its current state, its heuristic is the cost of the goal state compared to its current state. This is an admissible heuristic as the cost of the heuristic will always be less than or equal to the path cost

- Steps and Frequency: This heuristic depends on the number of differences in states compared to the goal state. For each index that the current state word is not equal to that same index in the goal state word, differences are increased. That combined with the heuristic that ensures the agent prioritizes states that point towards the goal state allow it to act in an optimized manner.

Difference (int) = 0

For each letter in the current word state:

If letter in current word state != same index of the goal word state:

Increment the number of differences

Return the number of differences when each letter has been assessed

- Scrabble: This heuristic also utilizes the goal state, however in a different manner than the steps and frequency heuristics. Each letter has their own "scrabble value" that dictates a variable cost that varies from letter to letter. Similar to the previous heuristics, this compares each index of the current state string and compares it to the current index of the goal state string. Unlike the previous heuristic however, if any index does not match, the cost of the next move is incremented by the scrabble value of the current index being assessed. If there is a match in an index, the heuristic cost for that level does not increase.

Cost (int) = 0

For each letter in the current word state:

If letter in current word state != same index of the goal word state:

Cost += scrabble_value of the current index of the goal state

Return the cost when each letter has been assessed

2. Explain what unavoidable remaining costs of lower-bounds.

Assuming the initial and goal states have different letters and letter sequences:

- Steps: Since each discrepancy only increases the differences uniformly, the unavoidable remaining cost is one step for every index in the current state that differs from the goal state. This means that the agent is eventually going to have to change n number of letters, where n is the length of the word, which is the lowest unavoidable cost.
- Frequency: With the cost of an action being $1 + (\text{rarity of current word})$, it is possible that the path cost can vary despite using similar formulas to the Steps heuristic. In a similar manner, the agent will eventually have to change n number of characters, where n represents the word size. Therefore the lowest unavoidable cost is $1 + (\text{rarity of goal word})$.
- Scrabble: Like the previous heuristics, the agent will eventually have to change n number of letters to reach the goal state. The difference however is that each letter in the goal word has an associated cost, with some letters having a larger cost than others. The lowest unavoidable cost is the sum of the scrabble values for each letter in the goal state.

3. Explain why it cannot overestimate the true optimal remaining cost. Your explanation must be specific to the corresponding cost measure. Merely stating that the heuristic “is a lower bound” is insufficient.

- Steps: Because each character replacement has a cost of one, the number of differing positions represents the minimum number of actions that must occur. A valid path may require additional changes as an intermediate result must also be a valid dictionary word, but it can never require fewer changes than the number of differing positions.
- Frequency: Frequency uses the replacement cost of $1 + \text{rarity}(w)$. Since each rarity value is > 0 , for n different positions, every possible solution must have a remaining cost of at least n , regardless of any extra letter replacements needed. These unavoidable replacements are counted by the heuristic, so it cannot overestimate the remaining optimal cost.
- Scrabble: Every position that differs from the goal must eventually be changed to its corresponding goal letter. Because the measure associates a cost with each letter, each scrabble value of the letters of the goal is unavoidable, and that cost must be paid at some point in the search. The optimal solution path cannot be formulated without paying the scrabble value of every mismatched goal letter.

Question 2: Does the heuristic actually help?

Run Uniform Cost and A* on

- WHY → ASK
- BEAR → DUCK

Run each program using all three cost measures. Calculate the percentage rendition as:

$$100 * (\text{UCS expansions} - \text{A* expansions}) / \text{UCS expansions}$$

Why → Ask

	Step	Scrabble	Frequency
UCS	773	765	808
A*	16	13	755

Bear → Duck

	Step	Scrabble	Frequency
UCS	691	2981	1084
A*	6	6	720

Percentages

	Step	Scrabble	Frequency
Why → Ask	97.9301423027	98.3006535948	6.55940594059
Bear → Duck	99.1316931983	99.79872526	33.5793357934

1. Did UCS and A* always return the same optimal cost? If not, identify and investigate the discrepancy

USC and A* had each returned optimal and equal solution costs. Due to their pathing rules however, each search algorithm had expanded different numbers of nodes, with A* expanding fewer nodes than UCS in most cases.

2. For each word pair, under which cost measure did A* produce the largest percentage reduction in expanded nodes?

Why → Ask: Scrabble had the largest reduction (98.30%)

Bear → Duck: Scrabble had the largest reduction (99.80%)

3. For each word pair, under which cost measure was the least informative?

Why → Ask: Frequency was the least informative cost measure since A* only reduced the number of node expansions by 6.559% compared to UCS. This is significantly smaller than the reductions from the step and scrabble heuristics, being 97.93% and 98.3% respectively. This signifies that the frequency heuristic provided much less useful guidance towards the goal.

Bear → Duck: Frequency was also the least informative cost measure, with A* reducing node expansion by 33.579% compared to UCS. This was also significantly smaller than the reductions achieved by step and scrabble heuristics, being 99.132% and 99.799% respectively.

In both cases, the frequency heuristic provided considerably less guidance to A*.

4. Explain the differences using the definitions of heuristics, not runtime alone (Different optimal paths are acceptable when their total cost are equal)

A heuristic defines rules to retain an estimated cost, or a guess. In the case of UCS, the heuristic is the smallest cost from the current node to the next node. This process allows the search algorithm to eventually reach the goal node and formulate an optimized path once the entire search tree has been expanded. UCS utilizes $g(n)$, the actual cost of a path from the starting node to the current node n .

A* uses this heuristic and expands upon it. While both search algorithms know when it expands upon the goal state, the heuristic for A* utilizes $h(n)$: an estimated cost from the current node to the goal node, on top of the $g(n)$ heuristic already described via UCS. This allows it to favor paths in a search tree that are more likely to reach a goal node. It is this combination of $g(n)$ and $h(n)$ that allows A* to track its current progress and use that to make a reasonable estimate on the future, where a goal node may lie.

Question 3: Reversibility and Direction-Dependent costs

DOGCAT actions are reversible: if one legal word can be changed into another, the reverse change is also legal. For each cost measure, determine whether the optimal cost from W1 to W2 must always equal the optimal cost from W2 to W1

For each cost measure:

1. State **yes** or **no**
2. Justify your conclusion
3. If the answer is no, give a concrete counterexample using words from the provided dictionary

Clearly distinguish between the reversibility of an action and the symmetry of its cost

- Steps: Yes, the cost measure is reversible. This is because the heuristic for a step cost measure is uniform. Because each path is equally weighted and each word is in the dictionary, there is no further optimized path for the search algorithm to favor. Notice when reversing the parameters in the program, the optimized solution path is the same.

```
[iy69135@linux1 PA1]$ python dcsolve.py why ask steps
dc(why, ask, steps) cost:5.00; time:0.004; solution:why wha aha ahs ass ask; deltas:[-2, -1, -1, 0.0, 0.0]
[iy69135@linux1 PA1]$ python dcsolve.py ask why steps
dc(ask, why, steps) cost:5.00; time:0.007; solution:ask ass ahs aha wha why; deltas:[-2, -1, -1, 0.0, 0.0]
[iy69135@linux1 PA1]$
```

- Frequency: No, the cost measure is not reversible. This is because the formula $1 + \text{rarity}(w)$ assigns a cost based on the word being entered. While the actions themselves are reversible, their costs are not always symmetric. When moving from word_a to word_b, entering word_b comes with a cost of word_b's rarity. When traveling from word_b to word_a, the cost comes from word_a's rarity. Since the cost of word_a and word_b may not always be equal, the solution path cost may not be equal, even if the solution path seems to reverse the sequence of word.

```
[iy69135@linux1 PA1]$ python dcsolve.py dog cat frequency
dc(dog, cat, frequency) cost:23.02; time:0.369; solution:dog don dan can cat; deltas:[-20.022, -13.737, -8.354, -6.887]
[iy69135@linux1 PA1]$ python dcsolve.py cat dog frequency
dc(cat, dog, frequency) cost:21.96; time:0.668; solution:cat can dan don dog; deltas:[-18.955, -16.489, -11.106, -5.82]
[iy69135@linux1 PA1]$
```

- Scrabble: No, the cost measure is not reversible. This is because each letter has an associated cost. While the optimized search reverses the same words when the parameters are reversed, some letters in the words are more expensive to replace than others. For example, in the first command, the cost of transitioning the “b” to a “d” in the last step has a cost of 2. However, in the first step of the second command, the cost to replace the “d” with a “b” has a cost of 3. Because the range of costs varies across each letter, the solution path cannot be reversible.

```
[iy69135@linux1 PA1]$ python dcsolve.py bear duck scrabble
dc(bear, duck, scrabble) cost:11.00; time:0.002; solution:bear beak beck buck duck; deltas:[0.0, 0.0, 0.0, 0.0]
[iy69135@linux1 PA1]$ python dcsolve.py duck bear scrabble
dc(duck, bear, scrabble) cost:6.00; time:0.001; solution:duck buck beck beak bear; deltas:[0.0, 0.0, 0.0, 0.0]
[iy69135@linux1 PA1]$
```

Question 4: LLM Heuristic Audit

Give a generative AI system the prompt provided in Blackboard and submit:

1. The model used, the exact prompt, and its proposed heuristic. A complete conversion transcript is not required
 2. A precise formula or pseudocode for the proposed heuristic
 3. Determine whether the heuristic is admissible. Give either a general proof or a concrete counterexample
 4. If the proposal is invalid, make the smallest repair you can or replace it with a safe alternative
 5. Test the verified or repaired heuristic against h0 on the two frequency-cost problems from Question 2. Compare solution costs and numbers of expanded nodes, then conclude whether the evidence supports using the proposed heuristic
- Current Model: ChatGPT 5.6 Luna
 - Prompt used: “In DOGCAT, the initial and goal states are lowercase words of the same length—three or four letters—and every legal state must appear in the supplied dictionary. A legal

action changes exactly one letter. Under the frequency objective, entering word w costs $1 + \text{rarity}(w)$, where the nonnegative rarity value is stored in the dictionary. The baseline heuristic $h_0(n)$ is the number of positions in which state n differs from the goal. Propose a rarity-aware heuristic intended to be more informative than h_0 while remaining admissible. Define it precisely and justify your proposal. By “more informative,” we mean that the proposed heuristic should be at least as large as h_0 for every state and strictly larger for at least one state, without overestimating the true optimal remaining cost.”

- Proposed heuristic, defined by $h_R(n)$:

Let $d(n, g)$ be the Hamming distance from the current word n to the goal g . The key observation is that **every legal move costs at least 1**, and its additional rarity cost can also be bounded from below.

Define

$$r_g = \text{rarity}(g)$$

and

$$r_{\min \neg g} = \min\{\text{rarity}(w) : w \in D, w \neq g\}.$$

For $n \neq g$, propose

$$h_R(n) = d(n, g) + (d(n, g) - 1) r_{\min \neg g} + r_g$$

and $h_R(g) = 0$.

- Pseudocode:
 $\text{Heuristic} = (\text{distance from current node to goal}) + ((\text{distance from current node to goal}) - 1) * (\text{minimum rarity value across each possible next state}) + \text{rarity of the goal node}$
- Admissibility Analysis: A heuristic is admissible if $h_R(n)$ is less than or equal to the lowest actual cost: The provided heuristic is admissible.
 - $(d(n, g))$ represents a path from the current node to the goal that requires at least d moves.
 - $(d(n, g) - 1)r_{\min \neg g}$ represents the minimum number of moves for each offset letter, including the goal node. For each node in $(d(n, g) - 1)r_{\min \neg g}$ that is not the goal node, the rarity has to be the minimum rarity across all possible current expansions. Since one of the moves involves entering the goal, and the heuristic for exploring a goal node is 0, $(d(n, g) - 1)r_{\min \neg g}$ is the path the search algorithm must take.
 - $+ r_g$ is the rarity of the goal node as the final step, being the added cost of exploring the goal node.
- Heuristic verifiability test: Both tests across my own implementation and the model's implementation have landed on the same solution costs. However, the AI's model had appeared to expand less paths to its solution than my own implementation. This falls in line with the model's

proposed heuristic, as it can provide a more informative estimate of the remaining cost, allowing A* to avoid expanding certain states otherwise considered by my implementation.

Results from my own implementation:

```
why -> ask, cost=frequency
UCS:
808 paths have been expanded and 89 paths remain in the frontier
Solution cost: 38.85540652
A*:
755 paths have been expanded and 123 paths remain in the frontier
Solution cost: 38.85540652
```

```
bear -> duck, cost=frequency
UCS:
1084 paths have been expanded and 1528 paths remain in the frontier
Solution cost: 38.2402284
A*:
720 paths have been expanded and 1367 paths remain in the frontier
Solution cost: 38.2402284
```

Results from the AI model's implementation:

```
why -> ask, cost=frequency
UCS:
808 paths have been expanded and 89 paths remain in the frontier
Solution cost: 38.85540652
A*:
634 paths have been expanded and 213 paths remain in the frontier
Solution cost: 38.85540652
```

```
bear -> duck, cost=frequency
UCS:
1084 paths have been expanded and 1528 paths remain in the frontier
Solution cost: 38.2402284
A*:
301 paths have been expanded and 806 paths remain in the frontier
Solution cost: 38.2402284
```

AI Disclosure: The only thing utilizing AI-Generated code is the frequency heuristic described in question 4. Given the AI model's proposed heuristic, I had asked it to provide the code for ease of testing. Rest assured that my implementation is still provided in dc.py, and the AI-generated code has been included but commented out in the final submission, so only my

implementation will be submitted. There is also a large mutli-line comment indicating its implementation in the python file.